AquiIoT

Funcionamento do Sistema

Visão geral do projeto

O AquiIoT é um sistema IoT de baixo custo para localizar pessoas e objetos em ambientes fechados, onde o GPS não funciona bem. Cada pessoa carrega um chaveiro inteligente; cada setor da edificação tem um ESP32 que detecta os chaveiros próximos e mede o ambiente; uma Raspberry Pi recebe, guarda e exibe tudo em um painel web.

Projeto Integrador V — Engenharia de Computação
UNIPAMPA — Campus Bagé

Documento gerado em 26/10/2026

1. Objetivo

Identificar **em qual setor** (sala, corredor, recepção...) cada chaveiro se encontra, e acompanhar as condições do ambiente de cada setor. O sistema não busca coordenadas exatas dentro de uma sala, e sim o ambiente onde o chaveiro está.

```
Chaveiro 01 -> Recepção
Chaveiro 02 -> Corredor A
Chaveiro 03 -> Sala 02
```

Aplicações: hospitais, shopping centers, indústrias, universidades e prédios comerciais.

2. Componentes

Componente	Hardware	Papel no sistema	Código
Chaveiro	ESP8266 (ESP-01S)	Elemento móvel. Conecta no Wi-Fi do setor e avisa a presença a cada 5 s com o RSSI.	esp8266_chaveiro/
Setor	ESP32 + LDR, microfone, DHT11, buzzer	Ponto fixo. Cria o Wi-Fi local, registra chaveiros, lê sensores e envia ao backend.	esp32_setor/
Unidade central	Raspberry Pi	Backend Node/Express (API + banco) e front-end React (Vite). Publicada na internet via Tailscale Funnel.	branch feat/arquitetura-web
Painel	Navegador	Mostra localização, sensores, estado de conexão, histórico e alertas.	front-end React

3. Arquitetura

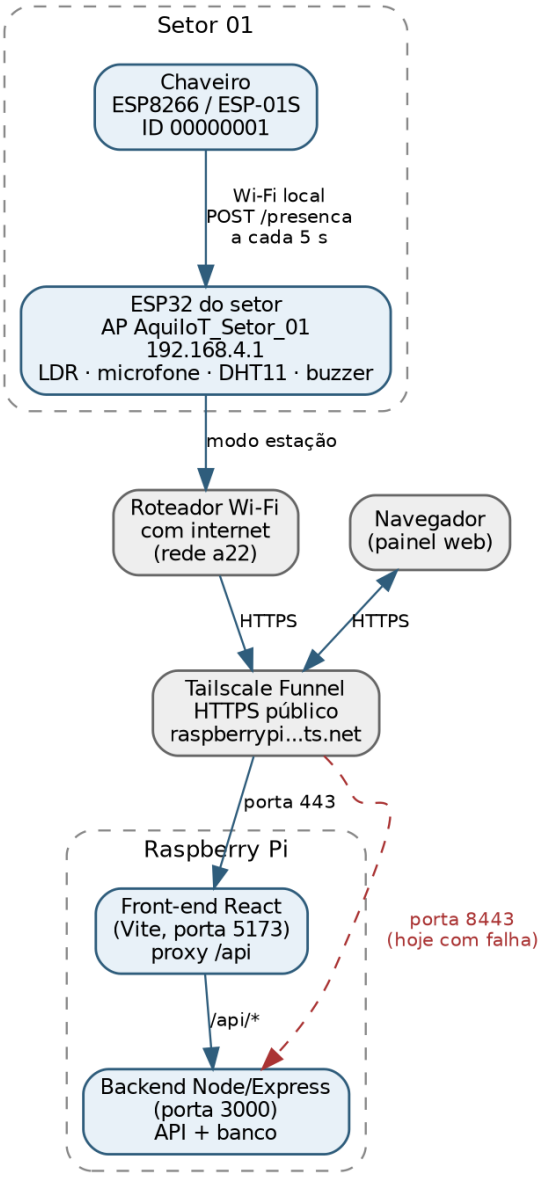


Figura 1 — Arquitetura do AquiloT

4. Caminho de uma informação de presença

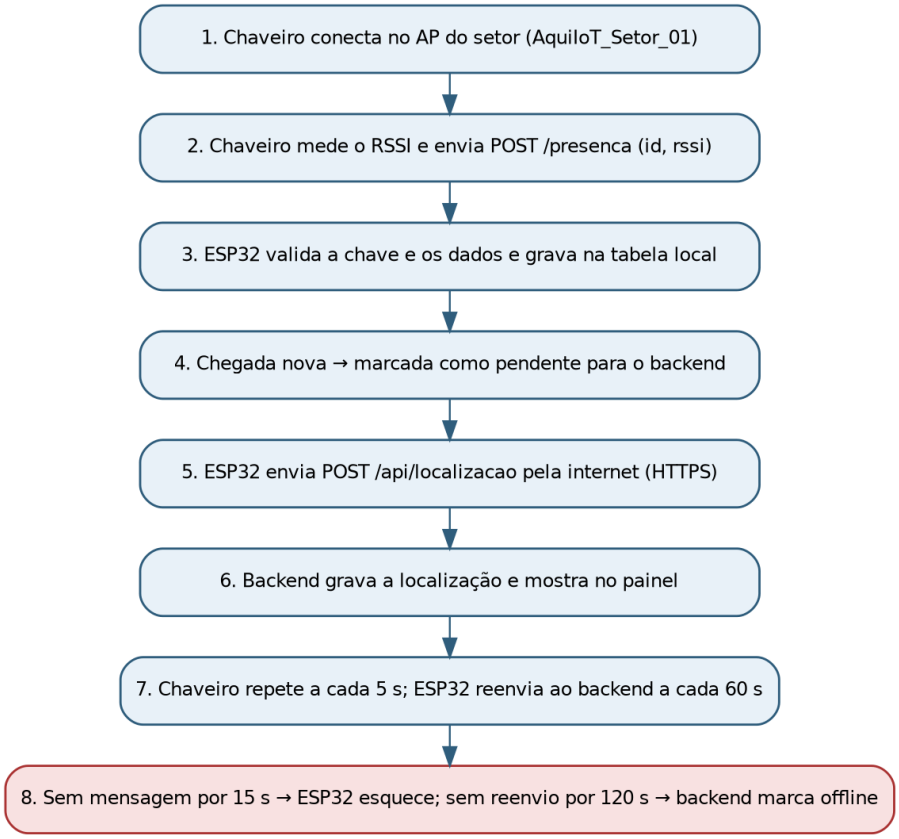


Figura 2 — Do chaveiro ao painel

4.1 Etapa local (chaveiro → ESP32)

O chaveiro conecta na rede `AquiIoT_Setor_01` e envia `POST http://192.168.4.1/presenca` com `id` e `rssi`, autenticado pelo cabeçalho `X-Device-Key`. O ESP32 guarda o chaveiro numa tabela em memória; se não ouvir o chaveiro por 15 s, ele deixa de contar como presente.

4.2 Etapa remota (ESP32 → Raspberry Pi)

Ao mesmo tempo, o ESP32 está conectado a um Wi-Fi com internet e envia ao backend, por HTTPS:

O quê	Rota	Quando
Cadastro do setor	POST /api/setores	Uma vez, ao conectar (201 criado ou 409 já existe).
Localização do chaveiro	POST /api/localizacao	Assim que o chaveiro chega e depois a cada 60 s enquanto continua no setor.
Leitura dos sensores	POST /api/eventos (tipo leitura_setor)	A cada 30 s.
Eventos do setor	POST /api/eventos (tipo setor)	Quando ocorrem (avisos, erros, recuperações, conexões).

O backend considera um chaveiro offline quando fica 120 s sem receber a localização dele.

5. Endereços e portas

Endereço	Destino	Situação
http://192.168.4.1	ESP32 do setor (rede local dos chaveiros)	Funcionando.
https://raspberrypi.tail656d5e.ts.net (443)	Vite (5173) → repassa /api para o backend	Funcionando. É o endereço usado hoje pelo ESP32.
https://raspberrypi.tail656d5e.ts.net:8443	Backend direto (3000)	Funnel ativo, mas a conexão fecha no handshake TLS.

Como o ESP32 usa a porta 443, o envio depende do servidor do Vite estar rodando na Raspberry Pi. Para o ESP32 falar direto com o backend, é preciso corrigir a porta 8443 do Funnel (por exemplo, `sudo tailscale funnel --bg --https=8443 3000`) e voltar `URL_BACKEND` para `...ts.net:8443`.

6. Segurança

- **Rede do setor**: WPA2 com senha (padrão 12345678 — trocar).
- **Chaveiro → setor**: chave compartilhada no cabeçalho X-Device-Key; ID com 8 dígitos e RSSI validados.
- **Buzzer**: chave separada no cabeçalho X-API-Key e limite de 1 bip por segundo.
- **Setor → backend**: HTTPS pelo Tailscale Funnel; o certificado ainda não é verificado no ESP32.
- **Segredos**: dados do Wi-Fi com internet e URL do backend ficam em `esp32_setor/segredos.h`, fora do Git.

7. Organização do repositório

```
AquiIoT/
├── Readme.md                    descrição do projeto
├── docs/
│   ├── AquiIoT_visao_geral.pdf  este documento
│   └── fontes/                  gerador dos PDFs e fluxogramas (Graphviz)
├── esp32_setor/                 firmware do ponto fixo (ESP32)
│   ├── esp32_setor.ino
│   ├── segredos.exemplo.h       modelo -> copiar para segredos.h
│   ├── segredos.h               (local, ignorado pelo Git)
│   └── docs/esp32_setor.pdf
└── esp8266_chaveiro/            firmware do chaveiro (ESP8266)
    ├── esp8266_chaveiro.ino
    └── docs/esp8266_chaveiro.pdf
```

8. Como colocar o sistema para funcionar

- **1.** Na Raspberry Pi, iniciar o backend (porta 3000) e o front-end (porta 5173) e conferir `tailscale funnel status`.
- **2.** Preencher `esp32_setor/segredos.h` e gravar o ESP32 do setor.
- **3.** Gravar cada chaveiro com um `ID_CHAVEIRO` diferente e a mesma rede/chave do setor.
- **4.** No Monitor Serial do ESP32 (115200), conferir “Setor pronto no backend” e “Presença de … enviada”.
- **5.** Abrir o painel web e verificar o chaveiro no setor.

9. Limitações atuais e próximos passos

- Apenas um setor implementado; o chaveiro conhece só a rede `AquiIoT_Setor_01`. Com vários setores, o chaveiro precisará escolher o de melhor sinal.
- Sensores de temperatura e luminosidade previstos para o chaveiro ainda não foram implementados.
- Leituras de sensores do setor vão ao backend como eventos (ainda não há tabela própria).
- Alto-falante de alerta na Raspberry Pi ainda não integrado.
- Corrigir a porta 8443 do Funnel, ativar verificação do certificado e trocar as chaves de exemplo.
- Conferir a ligação do LDR e do microfone (leituras fixas nos limites nos testes).

Apêndice — documentação de cada placa

Os detalhes de cada firmware (fluxogramas internos, rotas, temporizações, mensagens e gravação) estão em:

- `esp32_setor/docs/esp32_setor.pdf`
- `esp8266_chaveiro/docs/esp8266_chaveiro.pdf`